Rest API Todo List Manager

ECSE 429 Software Validation – Project Part A

Exploratory Testing and Unit Test Suite Report

Team member	Student ID	Email	Area tested
Yessine Chaari	261179816	yessine.chaari@mail.mcgill.ca	Todos endpoints
Shahzoab Shafi	261160765	shahzoab.shafi@mail.mcgill.ca	Projects endpoints

Application under test: runTodoManagerRestAPI-1.5.5.jar, run locally at http://localhost:4567.
October 2026.

1. Summary of Deliverables

The team tested the Rest API Todo List Manager, a service that stores todos, projects and categories and the relationships between them. As a two-person team we covered the todos and projects capabilities: each member ran one charter-driven exploratory session, logged the defects in a bug summary report, and built the automated unit tests for their area in one common JUnit suite.

Deliverable	Description
Exploratory session notes (PDF)	SessionNotes_Todos – Yessine Chaari, 29/09/2026, 42 notes. SessionNotes_Projects – Shahzoab Shafi, 10/10/2026, 43 notes.
Session scripts	Every session note has a replayable curl command (the "Program" section of each set of notes). `session/projects_session.sh` replays all 49 commands of the projects session, labelled by note number. `session/todoss_session.sh` replays all 50 commands of the projects session, labelled by note number
Bug summary reports	19 bugs in one common form (bugs 1–10 todos, 11–19 projects): executive summary of 80 characters or less, pre-conditions, description, potential impact, steps to reproduce, expected and actual result, severity, frequency, how it was found and the automated test that shows it.
Unit test suite	Java 21, JUnit 5, REST Assured and Maven. 143 tests in five test classes (TodosTest, ProjectsTest, KnownBugsTest, UndocumentedApiTest, CommandLineQueryTest).
Source code repository	https://github.com/ShahzoabShafi/rest-api-todo-list-manager-JUnit-test-cases Structure in section 4.
Unit test suite video	Part A unit test suite video (Google Drive). Shows the full suite running, the same seed twice (same order), a different seed (different order) and a run with the server stopped.

Deliverable	Description
Written report	This document.

Table 1: Deliverables.

Main result. The two sessions and the unit tests found **19 defects** (10 on todos, 9 on projects). The suite runs **143 tests: 124 pass and 19 fail**, and the 19 failures are exactly the 19 "expected-behavior" tests, one per bug. Every bug also has an "actual-behavior" test that passes and documents what the API really does.

2. Findings of Exploratory Testing

2.1 Approach

Both sessions followed the charter: identify the documented and undocumented capabilities of the API and its areas of potential instability, demonstrate each capability with a script, and exercise it with data typical of the intended use. Each session was timeboxed, started from a freshly started server with its default data, and used curl from the command line so that every observation is backed by a replayable command. Categories were only tested through their relationships with todos and projects, as the scope for a two-person team is todos and projects.

Session	Tester	Date and time	Duration	Notes	Bugs
Todos	Yessine Chaari	29/09/2026, 1:13–2:04 PM	51 min	42	1–6
Projects	Shahzoab Shafi	10/10/2026, 6:26–7:11 PM	45 min	43	11–19

Table 2: Exploratory sessions. The todos session ran 6 minutes over the 45-minute timebox.

2.2 Todos endpoints (Yessine Chaari)

Capabilities. CRUD on todos in JSON and XML; HEAD and OPTIONS; filtering of GET /todos by title and doneStatus; creating, querying and deleting todo/project (tasksof) and todo/category relationships; the undocumented Swagger file (/docs/swagger) and docs page (/docs); and request method override through the X-HTTP-Method-Override header (a defect, bug 6).

What worked. Most behavior matched the documentation: 201 on creation; 400 for a missing or empty title, a client-supplied id and malformed JSON; 404 for unknown ids and repeated deletion; 405 for PUT, DELETE and PATCH on /todos; no duplicate relationships; and 400 for a command injection attempt in a URL (/todos/1;ls).

What did not. An array title is silently truncated (bug 1); relationship endpoints answer 200 for a todo that does not exist (bugs 2, 3); POST /todos/:id/tasksof with an empty body creates and links a new empty project (bug 4); unsupported methods on /todos/:id/tasksof/:id return 404 instead of 405 (bug 5); and the method override header turns a forbidden PUT into a POST (bug 6).

2.3 Projects endpoints (Shahzoab Shafi)

Capabilities. CRUD on projects in JSON and XML (create, read, amend with POST and PUT, delete); HEAD and OPTIONS on /projects, /projects/:id and /projects/:id/tasks; filtering of GET /projects by title and active; creating, querying and deleting project/todo (tasks) and

project/category relationships; and an undocumented capability: POST /projects/:id/tasks with a todo body instead of an id creates a new todo that is already linked to the project.

What worked. 201 with a Location header on creation in JSON and XML; 400 for a client-supplied id, a non-boolean active field and malformed JSON or XML; 404 for unknown project ids and for deleting twice; 405 for PUT and DELETE on /projects and PATCH on /projects/:id; POST /projects/:id amends only the given field; deleting a project keeps its todos; project/todo links are visible from both sides and never duplicated.

What did not. PUT /projects/:id with only a title resets the other fields and removes all task links (bug 11). Projects have no title validation (bug 12) and an array title is truncated (bug 16). /projects/:id/categories answers 200 for an unknown project (bug 13), and a project/category link is only visible from the project side (bug 14). The method override header bypasses 405 on project endpoints too (bug 15). GET and POST on /projects/:id/tasks/:id return 404 instead of 405 (bug 17). A POST rejected for a non-boolean field still consumes an id (bug 18), and parse errors are sent with Content-Type text/html although their body is JSON (bug 19). Bugs 1, 5, 6 and 9 of the todos list reappear on projects, which suggests both entities share the same implementation.

2.4 Defects

Bugs 1 to 6 and 11 to 19 were found in the sessions. Bugs 7 to 10, and the DELETE variant of bug 15, were found while writing the unit tests, because the tests read the state back after every call. The full forms are in the bug summary reports.

#	Executive summary	Severity	Found in
1	POST /todos with array title silently keeps only the last element	Medium	Session
2	GET /todos/:id/tasksof returns 200 OK for a non-existent todo ID	Medium	Session
3	GET /todos/:id/categories returns 200 OK for a non-existent todo ID	Medium	Session
4	POST /todos/:id/tasksof with empty body links todo to every project	High	Session
5	GET/POST /todos/:id/tasksof/:id return 404 instead of documented 405	Low	Session
6	X-HTTP-Method-Override header turns a forbidden PUT into a POST (201)	High	Session
7	PUT /todos/:id with only a title resets doneStatus and description	Medium	Unit tests
8	PUT /todos/:id removes the todo's project and category relationships	High	Unit tests
9	GET /projects/:id/tasks returns 200 OK for a non-existent project ID	Medium	Unit tests
10	POST /todos/:id with empty tasksof array does not clear relationships	Low	Unit tests

#	Executive summary	Severity	Found in
11	PUT /projects/:id with only a title resets fields and drops task links	High	Session
12	POST /projects accepts a missing or empty title (201 Created)	Medium	Session
13	GET /projects/:id/categories returns 200 OK for a non-existent project	Medium	Session
14	Project-to-category link is not visible from /categories/:id/projects	High	Session
15	X-HTTP-Method-Override bypasses 405 on /projects endpoints (201)	High	Session + unit tests
16	POST /projects with an array title silently keeps only the last element	Medium	Session
17	GET/POST /projects/:id/tasks/:id return 404 instead of 405	Low	Session
18	A POST /projects rejected for an invalid boolean still consumes an id	Low	Session
19	Malformed JSON/XML errors are sent with Content-Type text/html	Low	Session

Table 3: Defects found (6 high, 8 medium, 5 low).

2.5 Concerns and areas of potential risk

- **Silent data loss.** PUT on todos and projects (bugs 7, 8, 11) wipes fields and relationships while returning 200, so a client that only renames an object loses data without being told.
- **Wrong information for unknown ids.** Four relationship endpoints answer 200 for an id that does not exist (bugs 2, 3, 9, 13), so clients cannot tell a missing object from an empty one.
- **Method override.** X-HTTP-Method-Override bypasses method restrictions on todos and projects (bugs 6, 15); a DELETE can even create a project. Proxies or authorization rules that filter on the HTTP method can be bypassed.
- **Weak and inconsistent validation.** Arrays are truncated (bugs 1, 16), projects accept an empty title while todos reject it (bug 12), and empty bodies create or link data (bug 4).
- **Inconsistent relationships and errors.** Project/category links are one-sided (bug 14); unsupported methods give 404 instead of 405 (bugs 5, 17); some errors have an empty body or a text/html content type (bug 19).

Not kept as defects. Unknown query parameters are silently ignored on GET /todos and GET /projects; this is undocumented and the suite accepts it. Error messages for malformed JSON are raw Java exceptions, but their 400 status is correct.

Not covered. Exhaustive malformed payloads on nested fields, XML bodies on relationship endpoints, combined filters, and the method override on every restricted endpoint.

3. Structure of the Unit Test Suite

The suite is a single Maven project (todo-api-tests) in Java 21 using JUnit 5 and REST Assured. It tests the running server at http://localhost:4567, which must be started before the tests. The infrastructure classes were written once and are shared by every test class.

Class	Tests	Role
BaseApiTest	–	Parent of every test class. Before each test it checks that the server answers and fails the test otherwise, then saves the system state; after each test it restores it. Provides `expectStatus` and `expectClientError`, which print and assert the status code of every request.
TodoApi, Payloads, Todo	–	HTTP helpers (GET, POST, PUT, DELETE, HEAD, OPTIONS in JSON and XML), setup helpers that create todos, projects and categories with unique titles, and a record used to compare todo state.
SystemSnapshot	–	Captures todos, projects, categories and their links; on restore it deletes everything a test created and repairs changed todos.
ResultReporter	–	JUnit extension that prints the random-order seed and PASS or FAIL after each test.
TodosTest	55	Documented todos endpoints: /todos, /todos/:id, /todos/:id/categories, /todos/:id/tasksof.
ProjectsTest	35	Documented projects endpoints: /projects, /projects/:id, /projects/:id/tasks, /projects/:id/categories.
KnownBugsTest	38	One pair of tests per bug (19 pairs): `bugN_expected_`… asserts the documented behavior and fails; `bugN_actual_`… asserts the real behavior and passes. Tagged `expected-behavior` and `actual-behavior`.
UndocumentedApiTest	10	Undocumented capabilities: OPTIONS, PATCH, unknown query parameters, Swagger and docs page (todos); OPTIONS, todo creation through /projects/:id/tasks and unknown parameters (projects).
CommandLineQueryTest	5	Runs curl as an external process to confirm command-line queries: filtering, HEAD and unknown ids for todos and projects.

Table 4: Classes of the suite (143 tests in total).

Test group	Tests	What is covered
/todos	15	Listing in JSON and XML, HEAD, filters, creation, invalid creation, malformed JSON and XML, 405.
/todos/:id	16	GET, HEAD, 404, amend with POST and PUT, empty title, malformed bodies, delete and repeated delete.

Test group	Tests	What is covered
/todos/:id/categories	11	Listing, link once, unknown category, malformed JSON, unknown todo, 405, delete links.
/todos/:id/tasksof	13	Same checks for projects, both link directions, 405 on /tasksof/:id, injection in the id.
/projects (A)	10	Listing in JSON and XML, HEAD, filters, creation in JSON and XML, id or invalid boolean rejected, malformed JSON and XML, 405.
/projects/:id (B)	11	GET in JSON and XML, 404, HEAD, amend with POST and PUT in JSON and XML, malformed or invalid amend, 404 on unknown id, 405, delete twice, deleting keeps todos and other projects.
/projects/:id/tasks (C)	8	Link once and visible on both sides (JSON and XML), HEAD, unknown todo or project, malformed JSON, 405, delete link on both sides, delete twice.
/projects/:id/categories (D)	6	Link once (JSON and XML), HEAD, unknown category or project, malformed JSON, 405, delete link twice.

Table 5: Test groups in TodosTest and ProjectsTest.

How the spec requirements are met.

- **Module per API.** One test group per documented endpoint (Table 5), plus UndocumentedApiTest for undocumented behavior and CommandLineQueryTest for command-line queries.
- **Expected and actual behavior.** For every bug, KnownBugsTest has a failing expected-behavior test and a passing actual-behavior test, so the API is shown to work in its undocumented way.
- **Application logic, not only status codes.** After each call the tests read the object back and check its fields and relationships, e.g. that a POST amend keeps the other fields and that a link is visible from both sides.
- **No unexpected side effects.** Tests compare counts before and after (e.g. a rejected POST creates nothing) and check that other objects are unchanged (e.g. deleting a project keeps its todos and the other projects).
- **JSON and XML.** Each endpoint is called with an XML request body or with Accept: application/xml at least once, and the XML response is parsed.
- **Return codes, malformed payloads and invalid operations.** Every request goes through `expectStatus`. Malformed JSON and XML, unknown ids, deleting twice and unsupported methods (405) are tested for each endpoint.
- **Ready, save, set up, execute, assess, restore.** BaseApiTest checks the server, takes a snapshot and restores it; each test creates its own data with unique titles and never modifies the default data, so tests are independent.
- **Fails if the service is not running.** The check in BaseApiTest fails every test with "Todo manager is not running at http://localhost:4567".
- **Random order.** `src/test/resources/junit-platform.properties` enables random order for both methods and classes, with a fixed seed so that a run can be repeated:

```
junit.jupiter.testmethod.order.default=org.junit.jupiter.api.MethodOrderer$
Random
junit.jupiter.testclass.order.default=org.junit.jupiter.api.ClassOrderer$Ra
ndom
junit.jupiter.execution.order.random.seed=20260930
```

Clean code. Test names state the behavior being checked (e.g. `deleteProjectKeepsItsTodosAndOtherProjects`), each test follows arrange–act–assert, and repeated steps live in small helpers (`linkTodo`, `linkCategory`, `withPostOverride`).

4. Source Code Repository

Repository:
https://github.com/ShahzoabShafi/rest-api-todo-list-manager-JUnit-test-cases

Folder / file	Description
pom.xml	Maven build (JUnit 5, REST Assured, Surefire)
README.md	How to run the suite, bug list, link to the video
src/test/java/api/	Test source code (11 files)
BaseApiTest.java	Checks the server is up, saves and restores state
SystemSnapshot.java	Captures and restores todos, projects, categories, links
TodoApi.java	HTTP helpers (JSON and XML)
Payloads.java, Todo.java	Request bodies, todo record
ResultReporter.java	Prints seed and PASS/FAIL per test
TodosTest.java	55 tests for /todos endpoints
ProjectsTest.java	35 tests for /projects endpoints
KnownBugsTest.java	38 tests: expected/actual pair for each of 19 bugs
UndocumentedApiTest.java	10 tests for undocumented behavior
CommandLineQueryTest.java	5 tests that run curl
src/test/resources/	Test configuration
junit-platform.properties	Random order and seed
session/	Exploratory session scripts
projects_session.sh	Replays the projects exploratory session
todos_session.sh	Replays the todos exploratory session
docs/	Session notes, bug summary reports, and the final report (PDF)

Run. Start the server with `java -jar runTodoManagerRestAPI-1.5.5.jar`, then run `mvn test` from the project root (or from the IDE). `mvn test`

`-Djunit.jupiter.execution.order.random.seed=12345` uses another order; `mvn test -Dgroups=expected-behavior` runs only the failing bug tests. The server keeps its data in memory, so a restart resets it.

5. Findings of Unit Test Suite Execution

The full suite was run with Maven on Ubuntu Linux (Dell XPS 15 7590, JDK 21) against a running server, as shown in the video.

Class	Run	Passed	Failed	Time	Note
TodosTest	55	55	0	10.2 s	
ProjectsTest	35	35	0	5.3 s	
KnownBugsTest	38	19	19	11.1 s	all 19 _expected tests
UndocumentedApiTest	10	10	0	1.6 s	
CommandLineQueryTest	5	5	0	0.8 s	
Total	143	124	19	30.6 s	seed 20260930

Table 6: Results of the run with seed 20260930.

Run	Seed	Class order (first test)	Result
1	20260930	KnownBugs, Todos, Undocumented, Projects, CommandLine (bug5_actual…)	143 run, 19 failed
2	20260930	Identical to run 1, test by test	143 run, 19 failed
3	12345	Todos, CommandLine, KnownBugs, Undocumented, Projects (deleteUnknownOrAlreadyDeletedTodoReturns404)	143 run, 19 failed
4	20260930, server stopped	Same as run 1	143 run, 143 failed

Table 7: Random-order runs. The same seed reproduces the same order; another seed changes it; the results do not depend on the order.

```
[ERROR] KnownBugsTest.bug11_expected_putWithOnlyATitle...:264 expected:
<keep me> but was: <>
[ERROR] KnownBugsTest.bug13_expected_categoriesOfUnknown...:313 expected:
<404> but was: <200>
[ERROR]   ...
[ERROR] Tests run: 143, Failures: 19, Errors: 0, Skipped: 0

run 4:  result: FAIL - Todo manager is not running at http://localhost:4567
[ERROR] Tests run: 143, Failures: 143, Errors: 0, Skipped: 0
```

Figure 1: Excerpt of the Maven output for runs 1 and 4.

Failures. The 19 failing tests are bug1_expected to bug19_expected. Each fails on the check described in its bug form, for example bug 13 expects 404 and receives 200, and bug 11 expects the description "keep me" and receives "". The matching 19 actual-behavior tests pass, so every defect is reproduced automatically in both directions.

Analysis. All 124 other tests pass in every order, so no failure comes from test setup or from one test depending on another. The automated tests found five issues the sessions missed: bugs 7 to 10 and the DELETE variant of bug 15, because they check state after each call and try every restricted method. The highest-impact defects are the silent data loss of PUT (bugs 8 and 11), the empty-body link of bug 4, and the method override (bugs 6 and 15). Bugs 7, 8 and 11 assume PUT should amend; if PUT is meant to replace the object, they become documentation issues.

Other observations. Field names in request bodies are case-insensitive (a "Description" field is stored as description), and boolean fields must be sent as JSON booleans but are returned as strings ("false"); neither is documented. While writing the tests we also learned that a test can pass for the wrong reason: an early rejected-POST test used a malformed body, so its 400 came from parsing rather than from validation. Each rejected-request test now uses a well-formed body. Bug 18 relies on ids being consecutive, which holds because the tests run one at a time.

6. Use of AI Tools

Shahzoab Shafi (projects). Claude (Anthropic) was used as a tutor and assistant. It explained JUnit 5 and REST Assured. It also turned the raw session observations into the session notes layout, formatted bug reports 11 to 19 in the team form, and reviewed the deliverables against the grading rubric.

Example prompts:

- Teach me how to use Junit
- Review my code. Make sure all the requirements from the project are met; if not, tell me exactly what I'm missing.

Yessine Chaari (todos). Claude was also used for formatting the raw session notes into a document with bullet points and different sections as well as formatting the todos bug report (1 to 10). It also helped formulate a solution for the tests to be independent of the order in which they are executed. It was also used to generate a table representing the source code structure for this report.

Example prompts:

- I gave you the txt file of my session notes for an exploratory test session, format it into a google docs document by adding appropriate numbering for each note, transforming keywords to sentences, and create a section where you order the commands that I used for each step.
- From the bugs identified in my session notes document, make a google docs template where I can fill in info about all the bugs I found in tables.
- How to make JUnit tests independent of the order in which they are executed
- can you do an image of the structure and a table with the nested structure too represented in it for google docs

All findings in this report were observed on the running application, and every test was run by the team.